RWTH AACHEN UNIVERSITY

Computer Vision – Lecture 14

Transformers for Vision II

26.06.2025

Bastian Leibe

Visual Computing Institute
RWTH Aachen University
http://www.vision.rwth-aachen.de/

leibe@vision.rwth-aachen.de

Course Outline

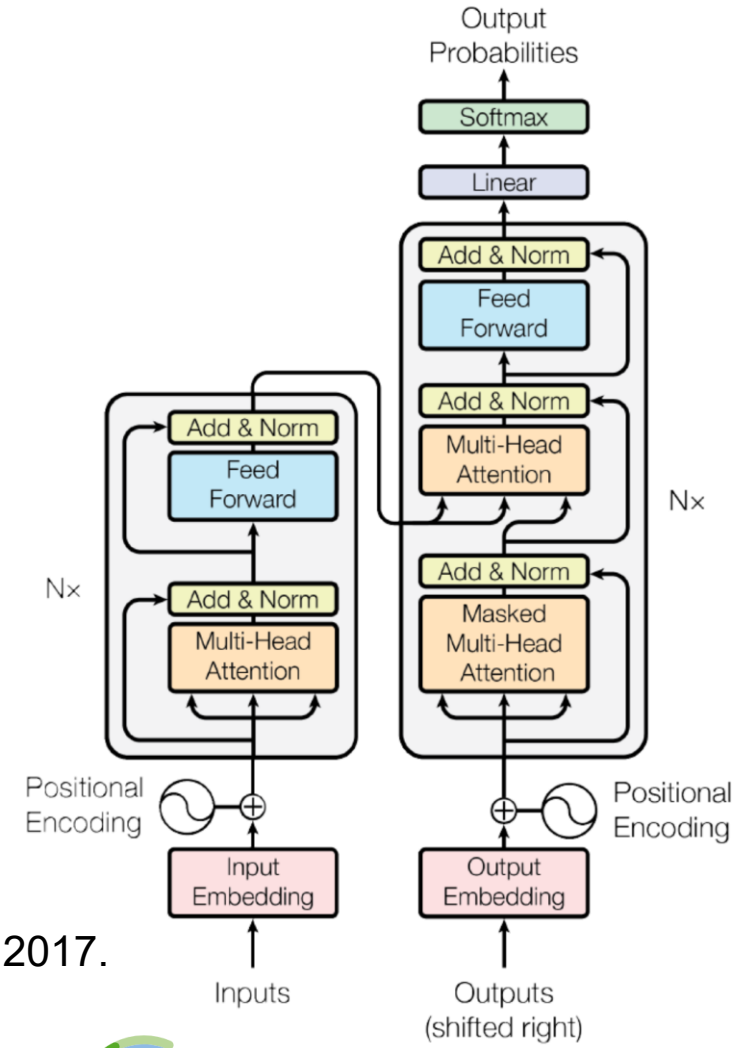
- Image Processing Basics
- Segmentation & Grouping
- Object Recognition & Categorization
  - Sliding Window based Object Detection
- Deep Learning for Vision
  - Convolutional Neural Networks (CNNs)
  - Deep Learning Background
  - CNNs for Object Detection
  - CNNs for Semantic Segmentation
  - CNNs for Matching
  - Transformers
- 3D Reconstruction

Topics of This Lecture

- Recap: Transformer Architecture
  - General Attention Layer
  - Self-Attention
  - Positional Encoding
  - Original Transformer Architecture
- Transformers for Vision
  - Image captioning with Transformers
  - Vision Transformers: ViT, Swin
  - Transformer Architectures

Recap: Transformers

- Transformer Architecture
  - Hugely influential recent development (2017)
  - Based on the idea of attention, but makes many improvements to concept of soft attention
  - Makes it possible to do seq2seq modeling *without* recurrent units
  - Most modern speech, language, and vision approaches are based on transformers nowadays…



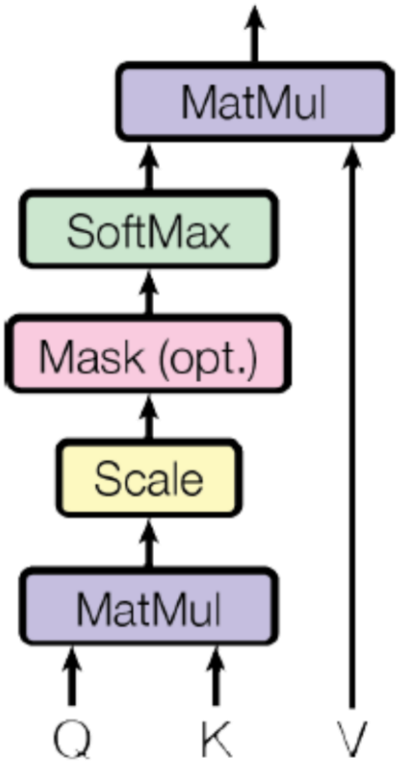
A. Vaswani et al., Attention is All You Need. NIPS 2017.

Image source: [Vaswani et al., 2017]

Transformer Components: Attention

- Scaled Dot-Product Attention
  - Transformer views the encoded representation of the input as a set of key-value pairs $(\mathbf{K}, \mathbf{V})$
  - In the decoder, the previous output is compressed into a query $\mathbf{Q}$
  - The next output is produced by mapping this query to the set of keys and values

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

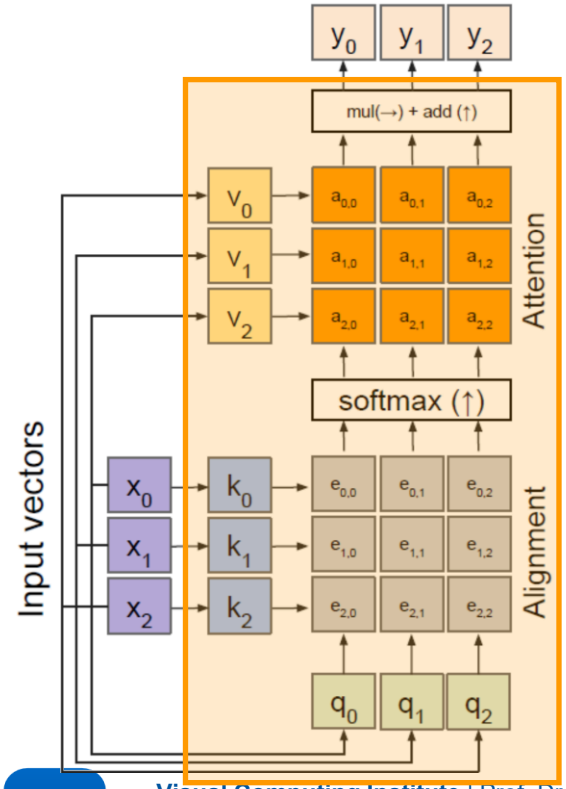


- Notes
  - Alignment is expressed via dot product (cosine similarity)
  - Normalization with scaling factor $d_{\mathbf{K}}$ (dimension of $\mathbf{K}$) is motivated by concern that the softmax function may have an extremely small gradient for large inputs

Image source: [Vaswani et al., 2017]

Transformer Components: Self-Attention Layer

- Self-attention layer attends over sets of inputs

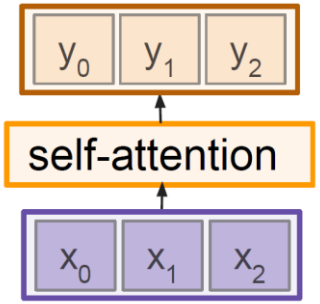


Outputs

- Context vectors: $\mathbf{y}$ (shape: $D_v \times 1$)

Operations

- Key vectors: $\mathbf{k} = \mathbf{x}\,W_k$
- Value vectors: $\mathbf{v} = \mathbf{x}\,W_v$
- Query vectors: $\mathbf{q} = \mathbf{x}\,W_q$:
- Alignment: $e_{i,j} = q_j \cdot k_i / \sqrt{D}$
- Attention: $\mathbf{a} = softmax(\mathbf{e})$
- Output: $y_j = \sum_i a_{i,j} v_i$

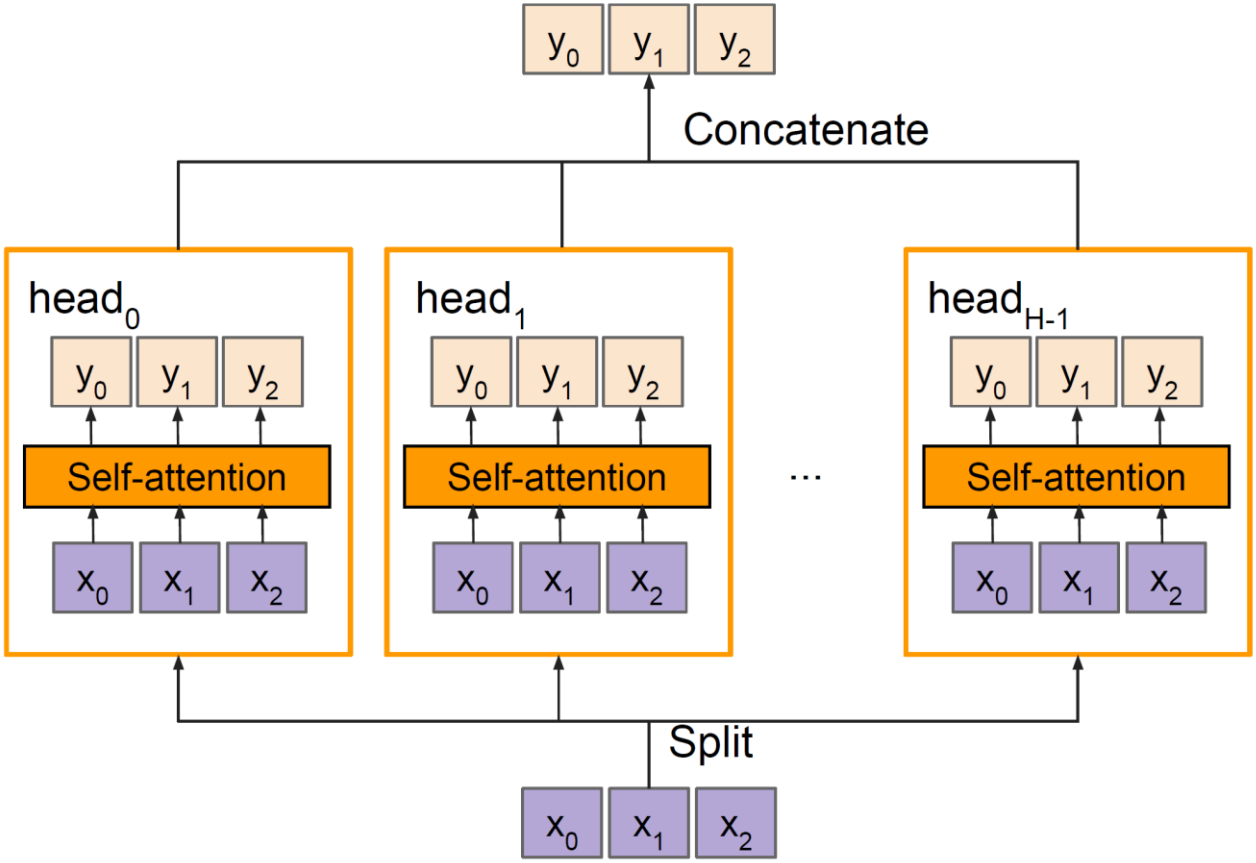


Inputs

- Input vectors: $\mathbf{x}$ (shape: $N \times D$)

Transformer Components: Multi-Head Self-Attention

- Multiple self-attention heads in parallel



Slide credit: Fei-Fei Li

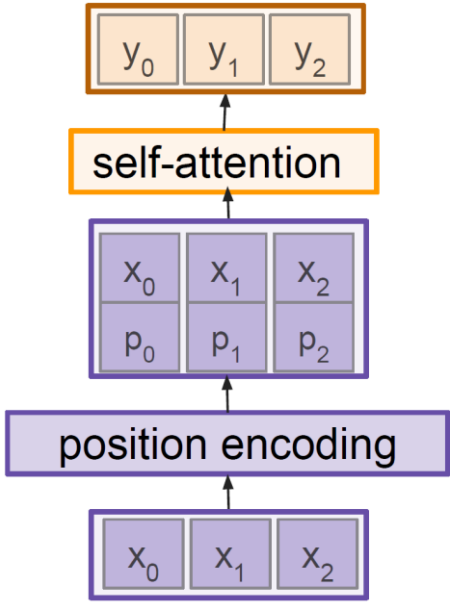
Transformer Components: Positional Encoding

- Idea
  - As attention is permutation invariant, concatenate / add a special positional encoding p_j to each input vector x_j
  - This encoding can be formulated as a function
    $$pos: \mathbb{N} \rightarrow \mathbb{R}^d$$
    that transforms the position j of the input into a d-dimensional vector $p_j = pos(j)$.
  - Design a function with the desired properties

$$p(t) = \begin{bmatrix} \sin(\omega_1 \cdot t) \\ \cos(\omega_1 \cdot t) \\ \sin(\omega_2 \cdot t) \\ \cos(\omega_2 \cdot t) \\ \vdots \\ \sin(\omega_{d/2} \cdot t) \\ \cos(\omega_{d/2} \cdot t) \end{bmatrix}$$

where

$$\omega_k = \frac{1}{10000^{2k/d}}$$



Topics of This Lecture

- Attention & Transformers
  - Motivation
- Transformer Architecture
  - General Attention Layer
  - Self-Attention
  - Positional Encoding
  - Original Transformer Architecture
- Transformers for Vision
  - Image captioning with Transformers
  - Vision Transformers: ViT, Swin
  - Transformer Architectures

Topics of This Lecture

- Recap: Transformer Architecture
  - General Attention Layer
  - Self-Attention
  - Positional Encoding
  - Original Transformer Architecture
- Transformers for Vision
  - Image captioning with Transformers
  - Vision Transformers: ViT, Swin
  - Transformer Architectures

Image Captioning using Transformers

- Input: Image $\mathbf{I}$
- Output: Sequence $\mathbf{y} = y_1, y_2, \ldots, y_T$

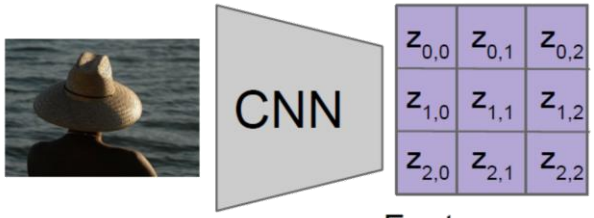


Image Captioning using Transformers

- Input: Image $\mathbf{I}$
- Encoder $\mathbf{c} = T_{\mathbf{W}}(\mathbf{z})$
  - where $\mathbf{z}$ are spatial CNN features, $T_{\mathbf{W}}(\cdot)$ is the Transformer encoder
- Output: Sequence $\mathbf{y} = y_1, y_2, \ldots, y_T$

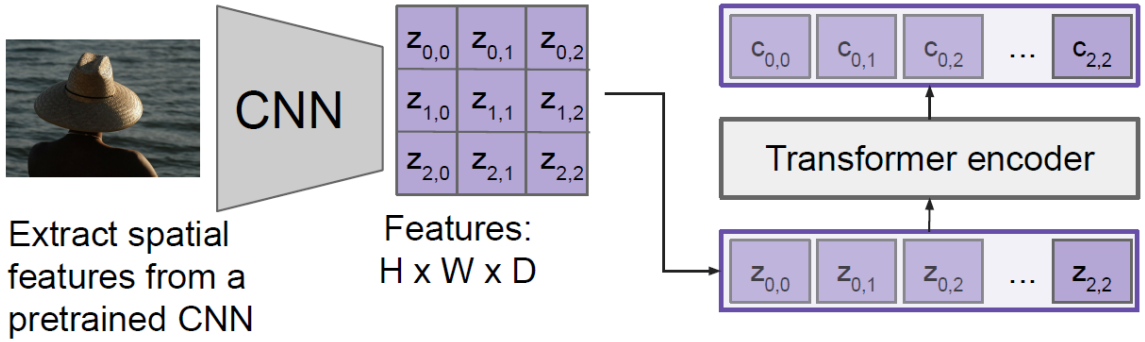
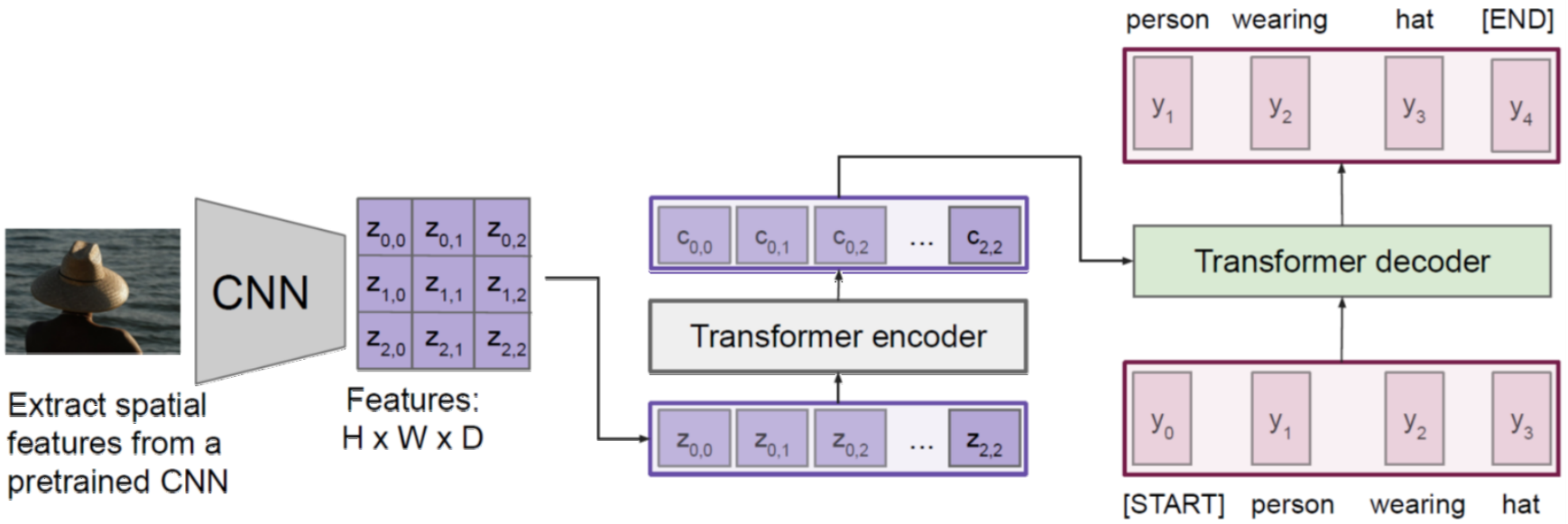


Image Captioning using Transformers

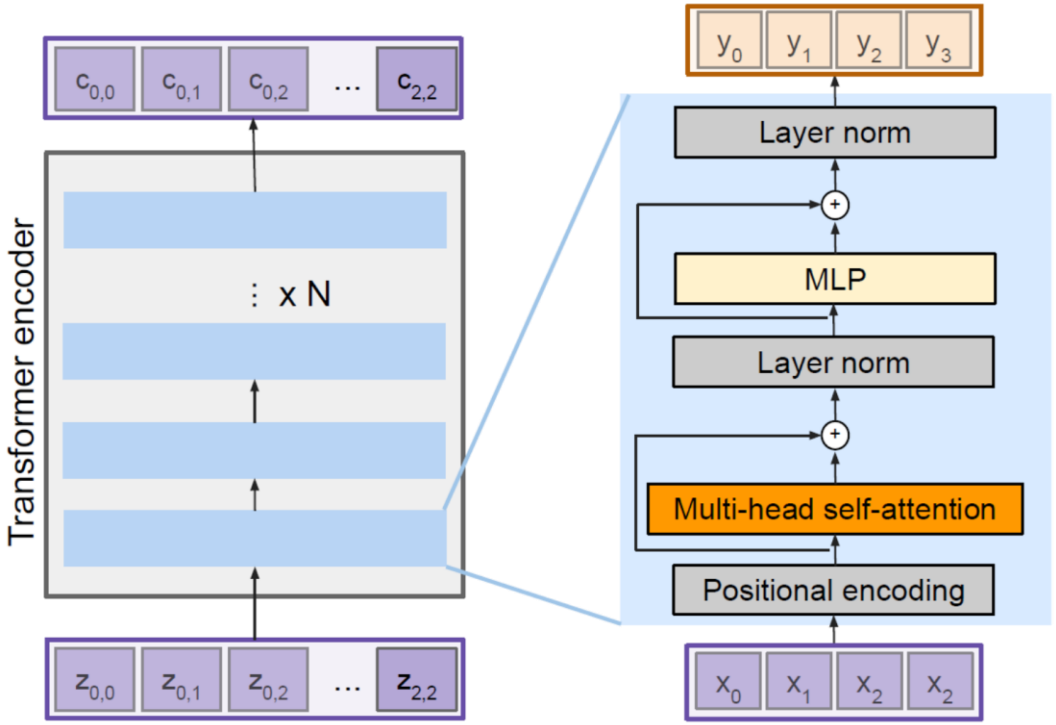
- Input: Image $\mathbf{I}$
- Encoder $\mathbf{c} = T_{\mathbf{W}}(\mathbf{z})$
  - where $\mathbf{z}$ are spatial CNN features, $T_{\mathbf{W}}(\cdot)$ is the Transformer encoder
- Output: Sequence $\mathbf{y} = y_1, y_2, \ldots, y_T$
- Decoder $y_t = T_{\mathrm{D}}(\mathbf{y}_{0:t-1}, \mathbf{c})$
  - where $T_{\mathbf{D}}(\cdot)$ is the Transformer decoder



Slide credit: Fei-Fei Li

Transformer Encoder

- Transformer Encoder block

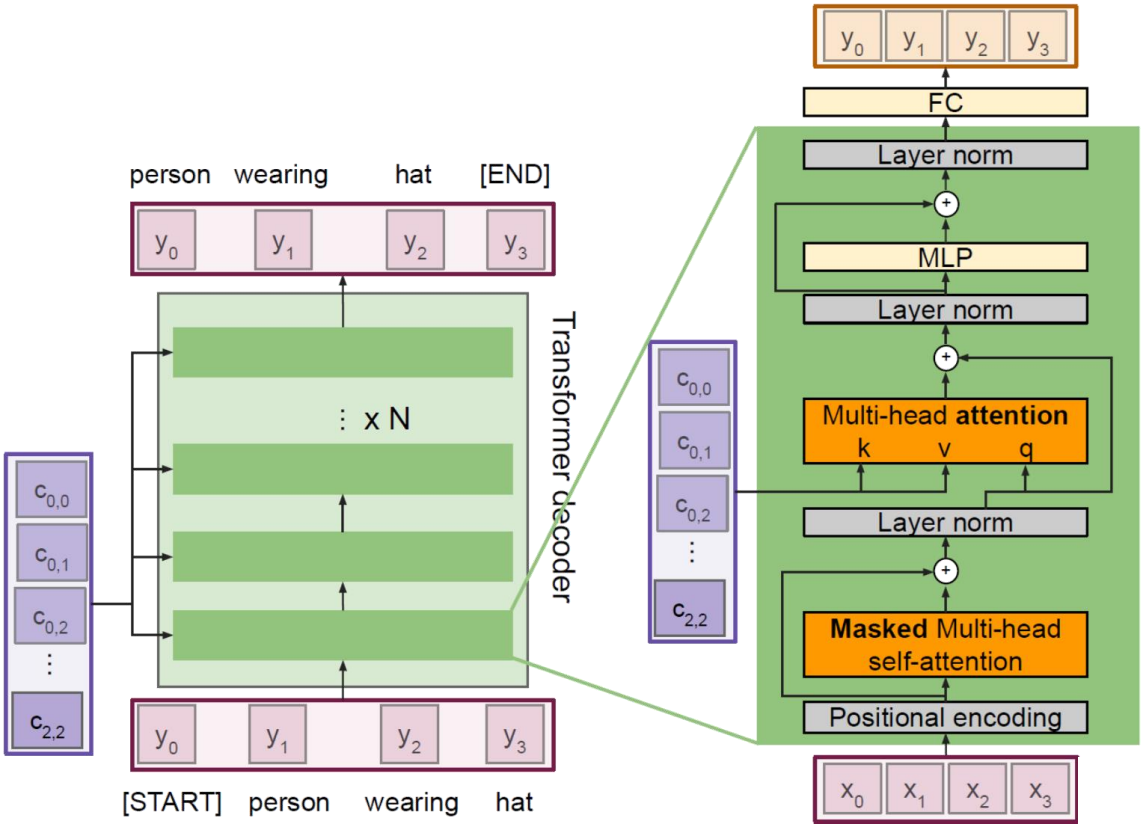


- Inputs: Set of vectors $\mathbf{x}$
- Outputs: Set of vectors $\mathbf{y}$
- Self-attention is the only interaction between vectors
- Layer norm and MLP operate independently per vector
- Highly scalable, highly parallelizable, but high memory usage

Image source: [Vaswani et al., 2017]

Transformer Decoder

- Transformer Decoder block

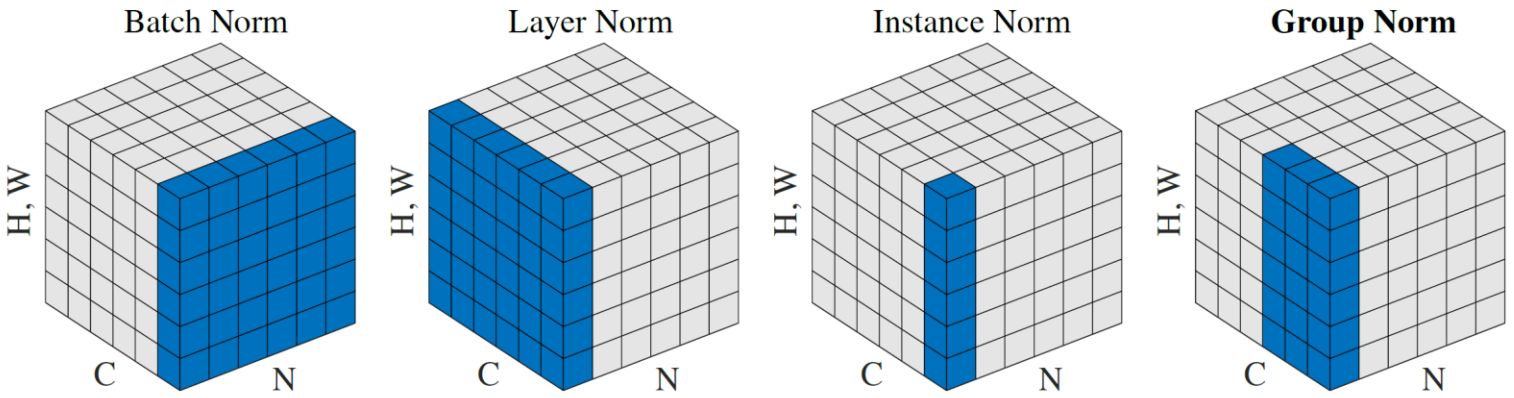


- Inputs: Set of vectors $\mathbf{x}$ and Set of context vectors $\mathbf{c}$
- Outputs: Set of vectors $\mathbf{y}$
- Masked Self-attention only interacts with past inputs.
- Multi-head attention block is NOT self-attention. It attends over encoder outputs.
- Highly scalable, highly parallelizable, but high memory usage

Slide credit: Fei-Fei Li

Image source: [Vaswani et al., 2017]

Layer Normalization



W width
H height
C #channels
N #examples

- Recall: BatchNorm
  - Standardizes all activations within a given feature channel to zero mean, unit variance
  - This helps significantly for training and allows for a larger learning rate
  - However, when batch size is small, the estimated mean and variance become unreliable
- Alternative norms
  - Instead of normalizing over examples in a minibatch, we can normalize over other dimensions of the activation tensor
  - Layer Norm normalizes over all activations from a single training example instead

Image from [Wu & He, Group Normalization, ECCV 2018]

Comparing CNNs, RNNs, and Transformers

- (Temporal) CNN
  - Kernel size k, d feature channels
  - Time to compute output for sequence of length n is $\mathcal{O}(knd^2)$, can be parallelized
  - Need stack of n/k layers
- RNN
  - Computational complexity is $\mathcal{O}(nd^2)$ for a hidden state of size d, inherently sequential.
  - Maximum path length is $\mathcal{O}(n)$.
- Self-attention
  - Every output is directly connected to every input, so maximum path length is $\mathcal{O}(1)$.
  - But computational cost is $\mathcal{O}(n^2d)$.
  - *This is a problem for long sequences or large inputs.*

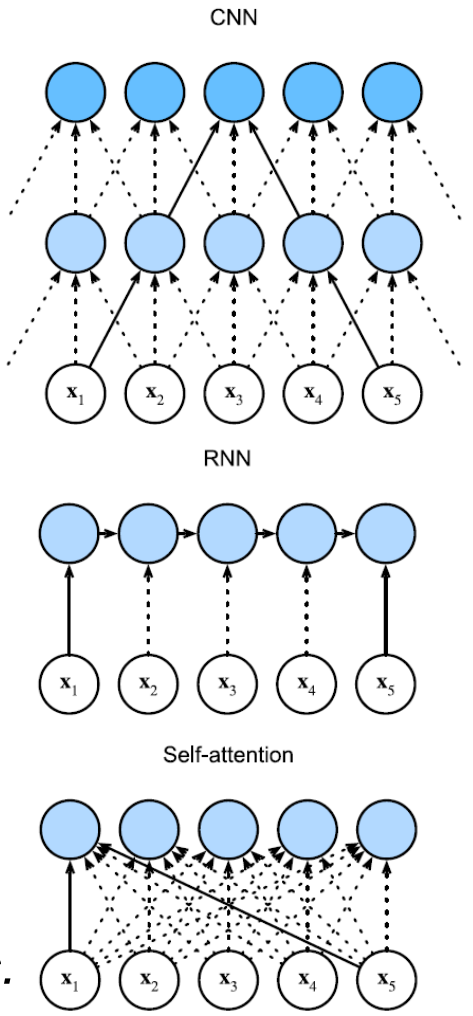
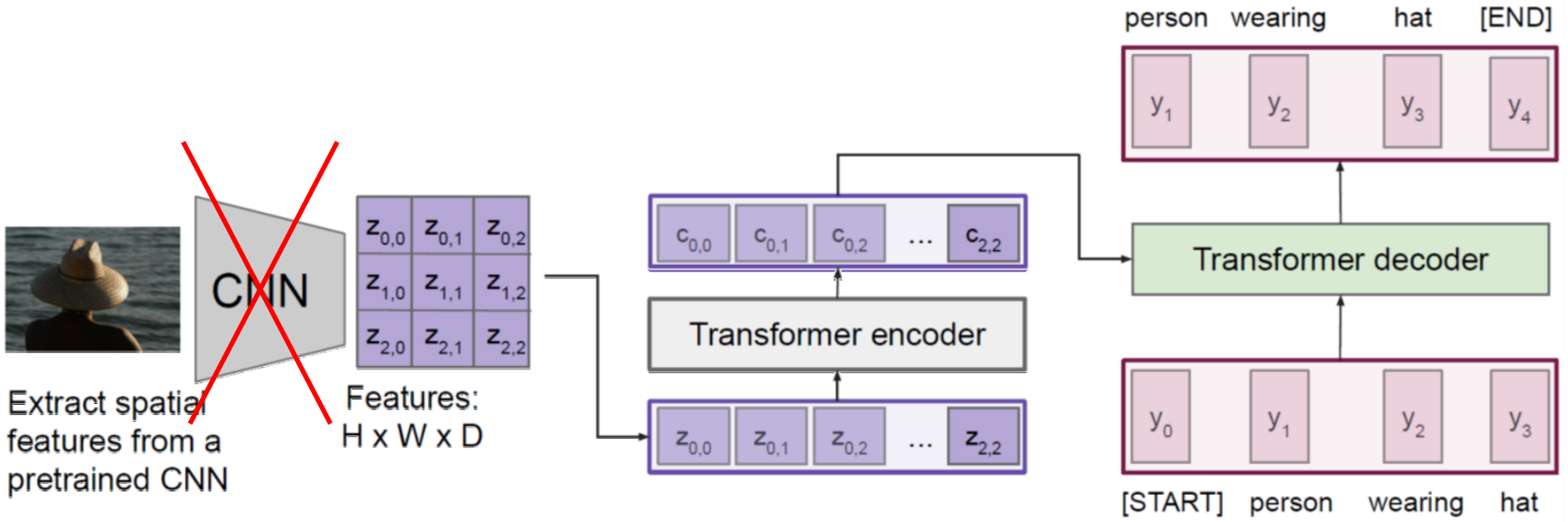


Figure from [Zhang et al., Dive into Deep Learning, 2020]

Image Captioning using Transformers

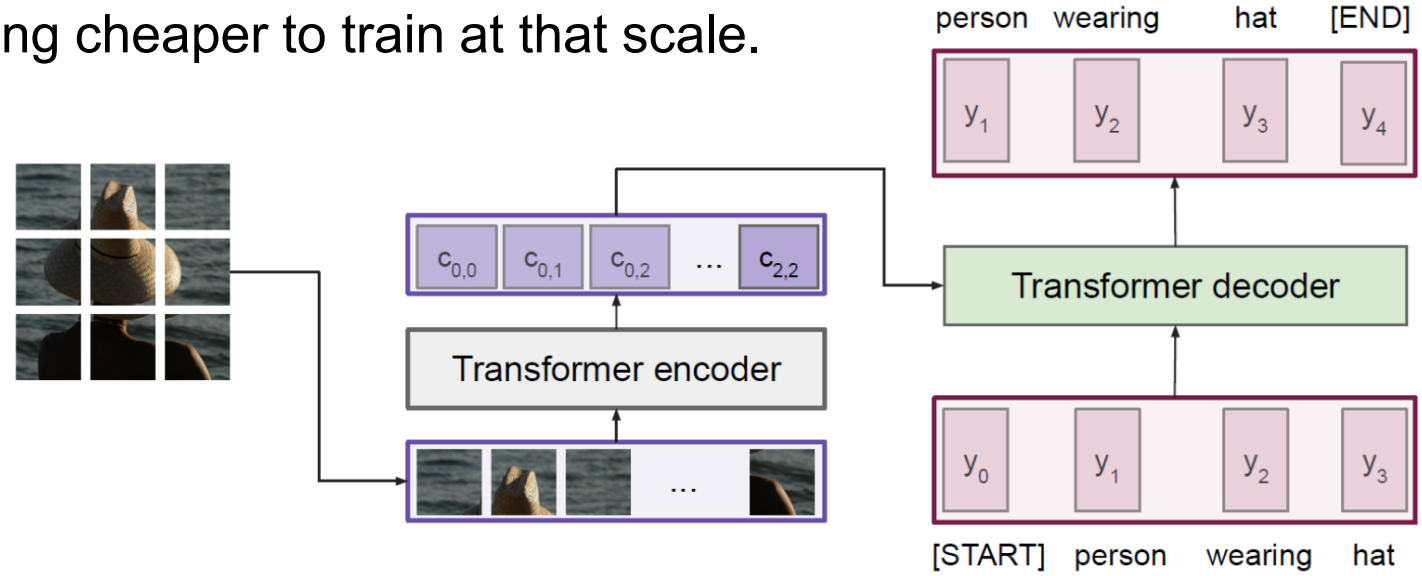
- Can we build a purely Transformer based vision model?
  - I.e., a model that doesn't use CNNs at all?



Slide credit: Fei-Fei Li

Image Captioning using ONLY Transformers

- Vision Transformer (ViT)
  - Chop up the image into 16×16 pixel patches
  - Project each pixel into an embedding space
  - Pass the set of embeddings $\mathbf{x}_{1:T}$ to a Transformer
  - When trained on large datasets (ImageNet21k with 14M images), ViT does better than a big CNN ResNet model, while being cheaper to train at that scale.



Slide credit: Fei-Fei Li

[Dosovitskiy et al., ICLR 2021]

Vision Transformer (ViT)

- Detailed view

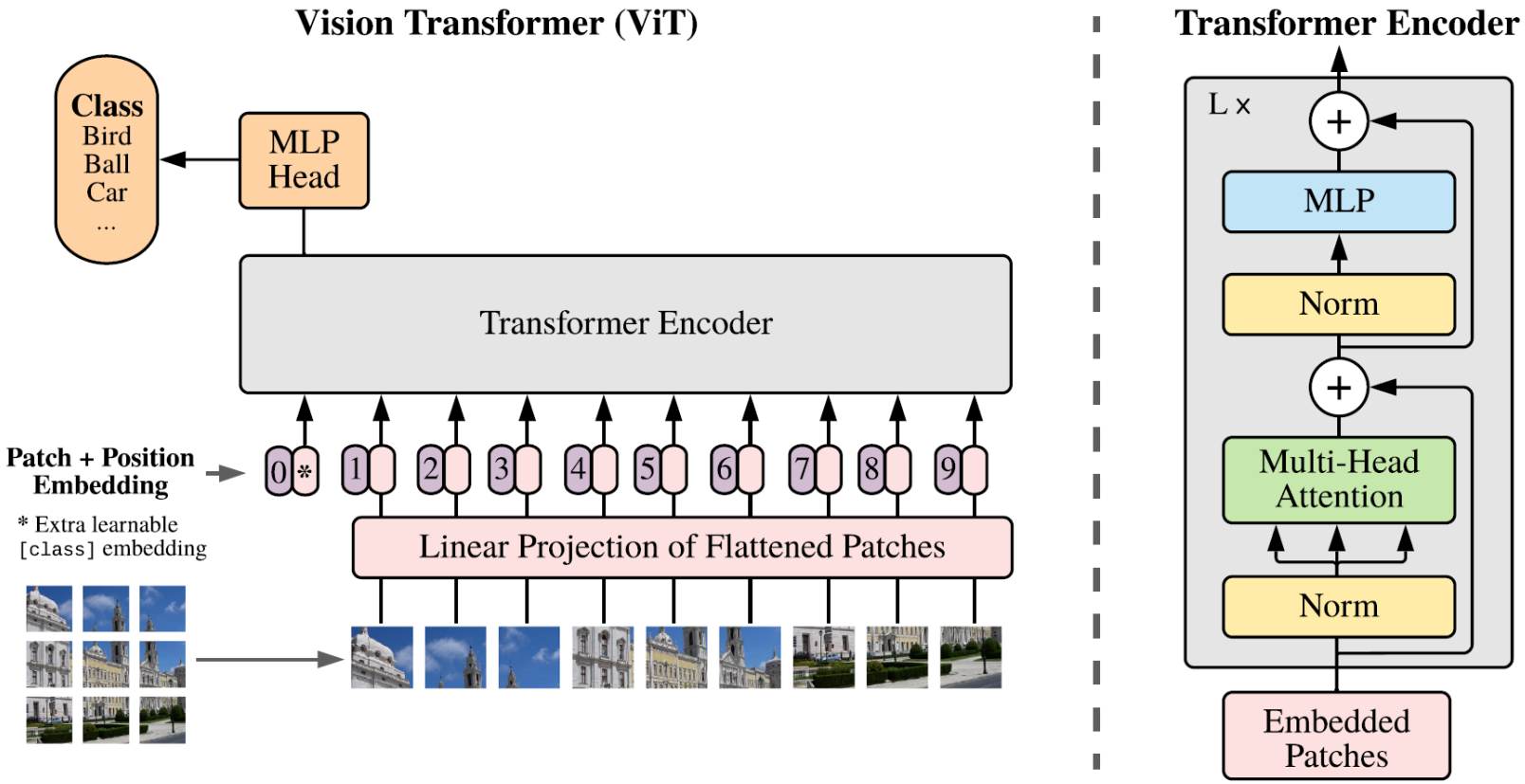


Figure from [Dosovitskiy et al., ICLR 2021]

Vision Transformers vs. ResNets

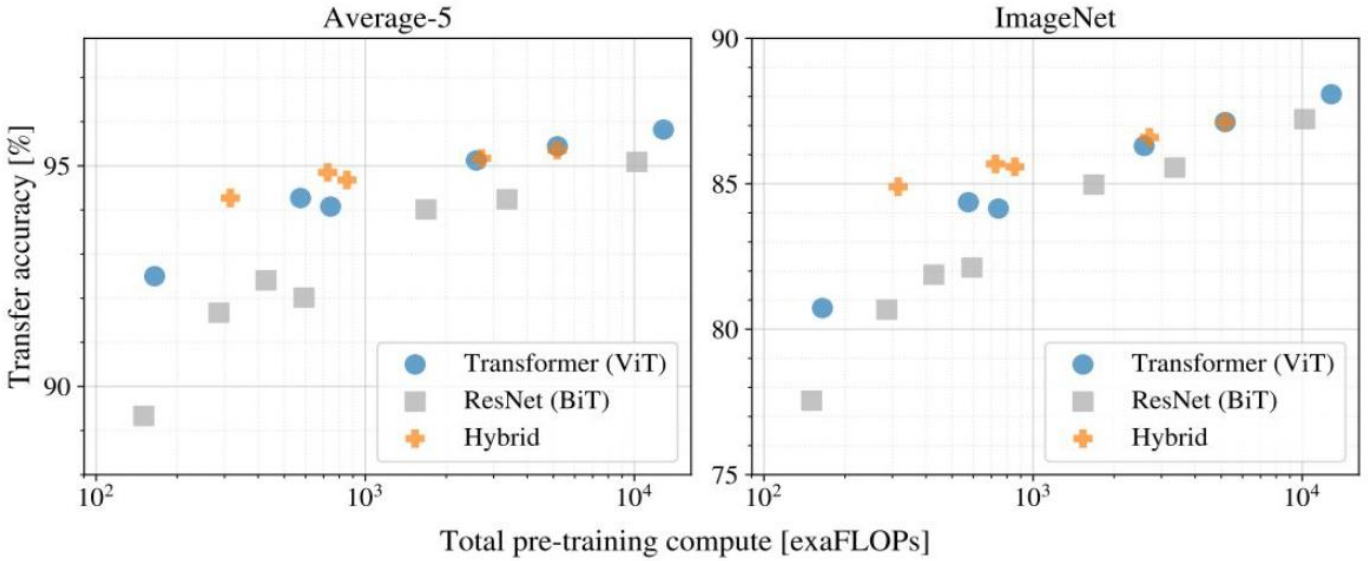
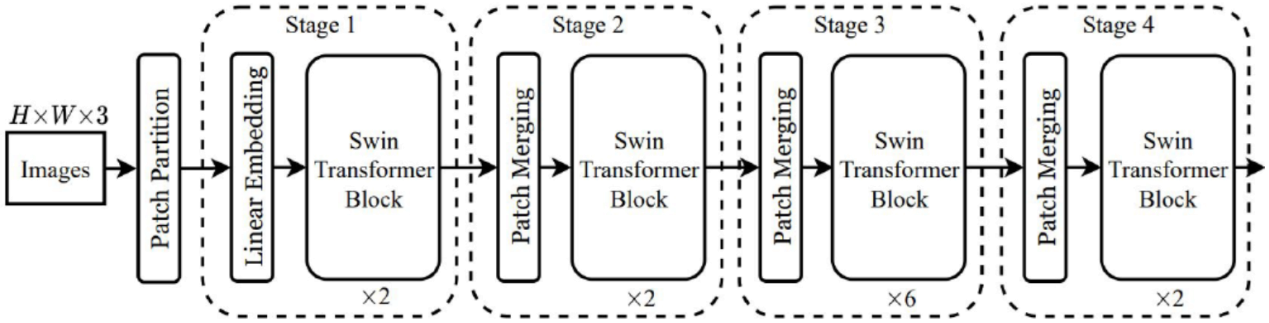


Figure 5: Performance versus cost for different architectures: Vision Transformers, ResNets, and hybrids. Vision Transformers generally outperform ResNets with the same computational budget. Hybrids improve upon pure Transformers for smaller model sizes, but the gap vanishes for larger models.

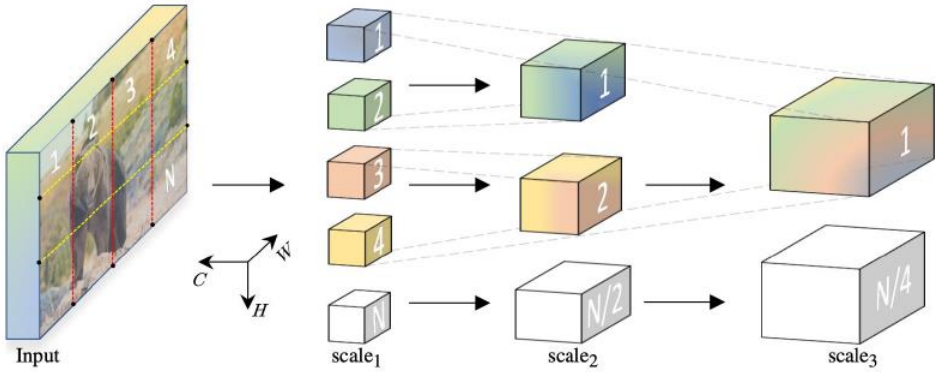
Slide credit: Fei-Fei Li

Source: [Dosovitskiy et al., ICLR 2021]

Vision Transformers: Extensions

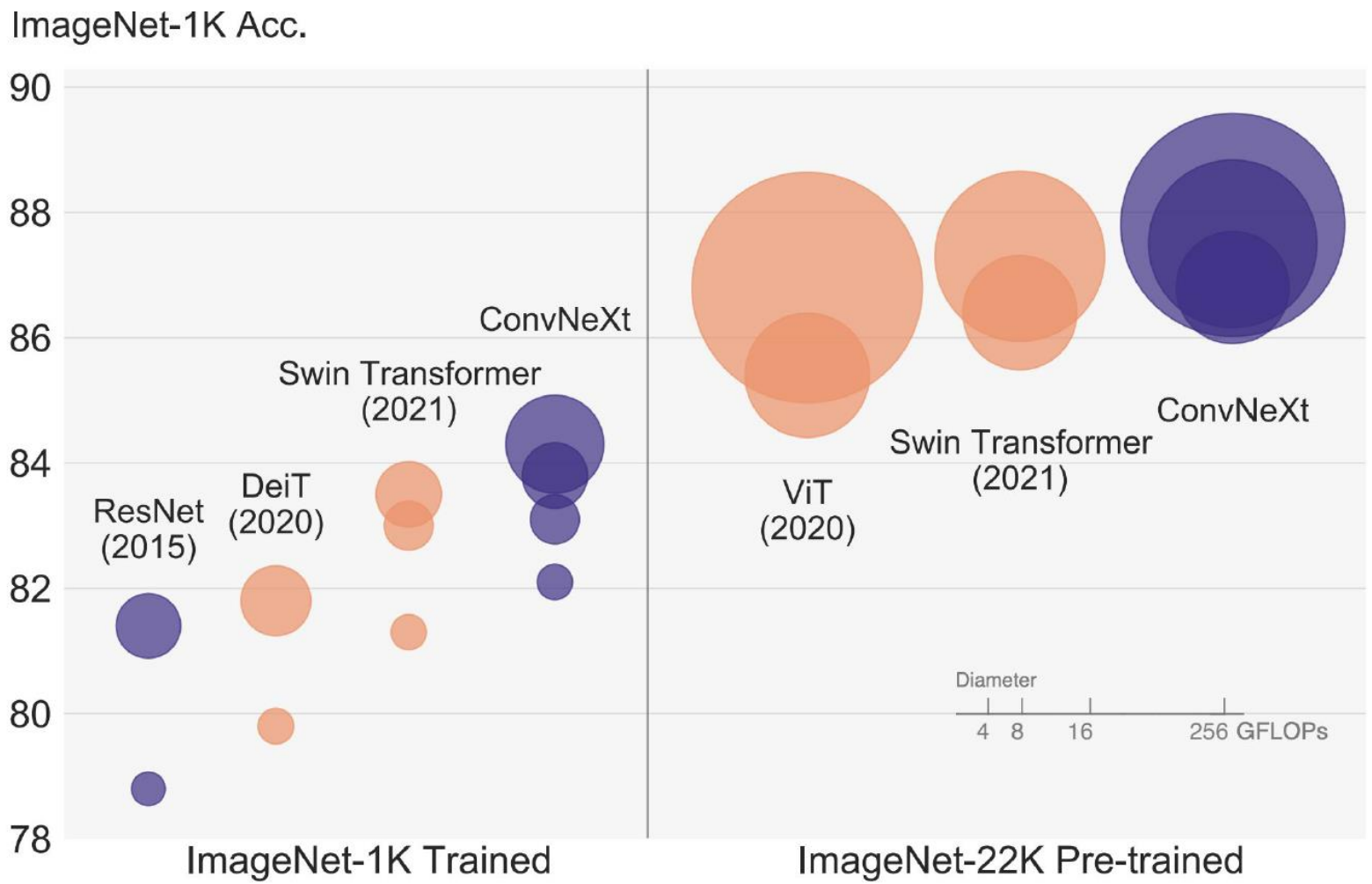


Liu et al, "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows", CVPR 2021



Fan et al, "Multiscale Vision Transformers", ICCV 2021

CNNs: The Empire Strike Back…



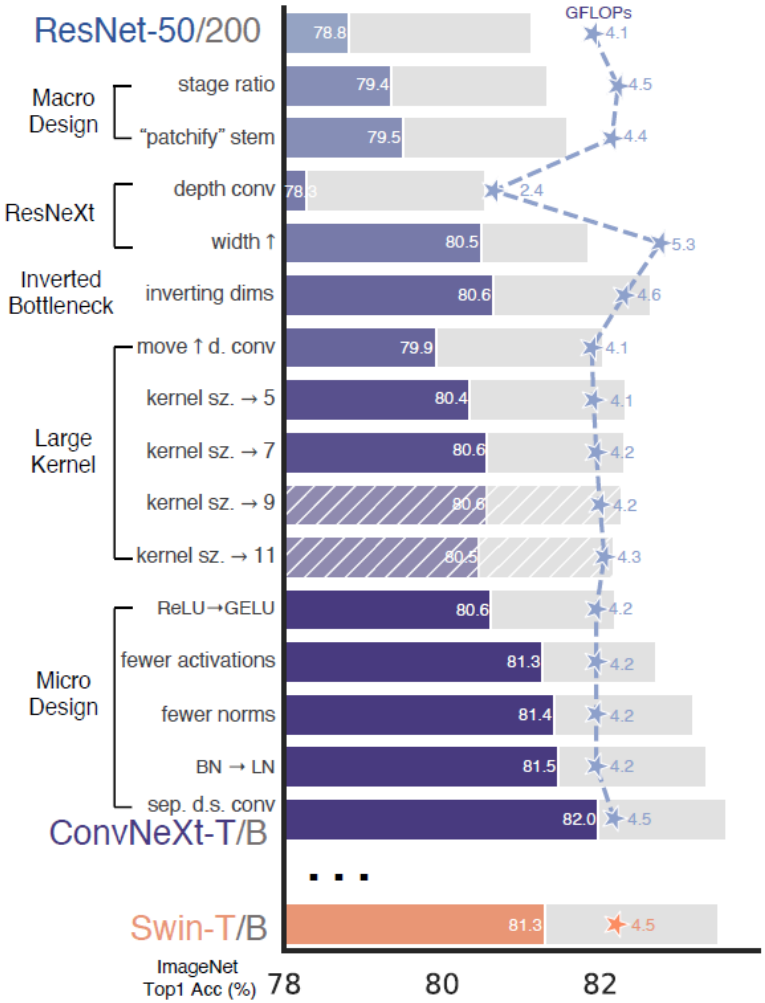
Slide credit: Fei-Fei Li

Source: [Liu et al., CVPR 2022]

CNN vs. Transformer

- Which one is best?
  - It turns out CNNs can also improved in many small ways…
  - "Modernized" CNN (ResNet) using design elements of a hierarchical vision transformer (Swin) again performs at a similar level
  - CNNs have a useful built-in inductive bias
    - Locality (small kernels)
    - Equivariance (due to weight tying)
    - Invariance (due to pooling)

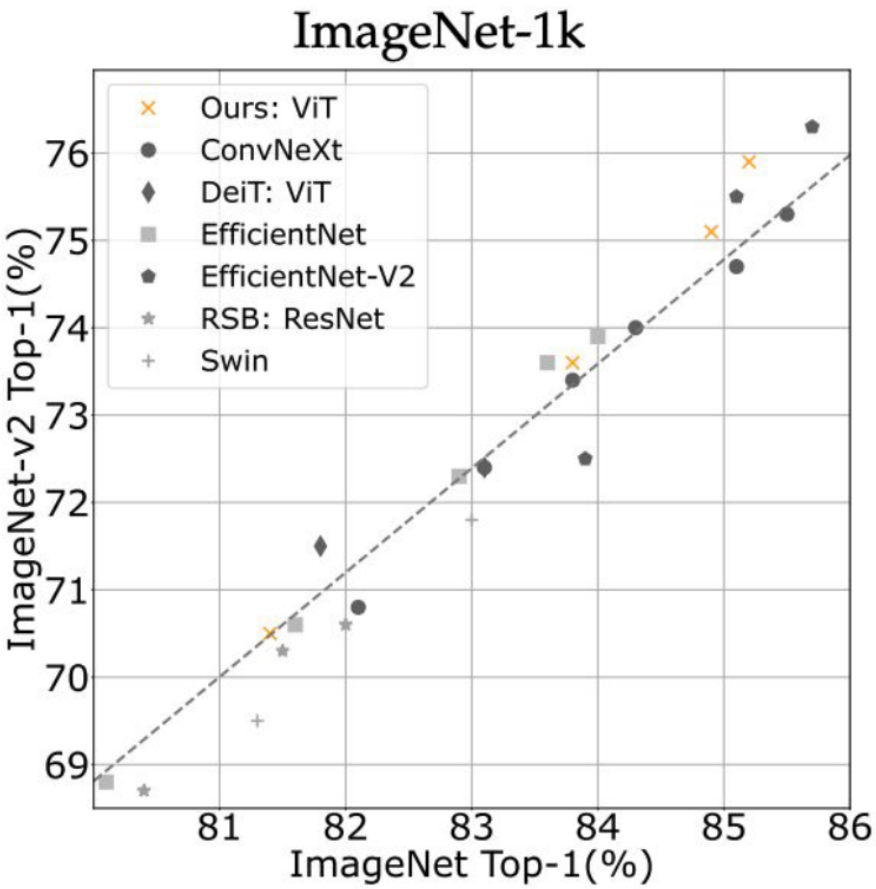
    ⇒ *CNNs will likely stay relevant, but the trade-offs when to use what will evolve.*

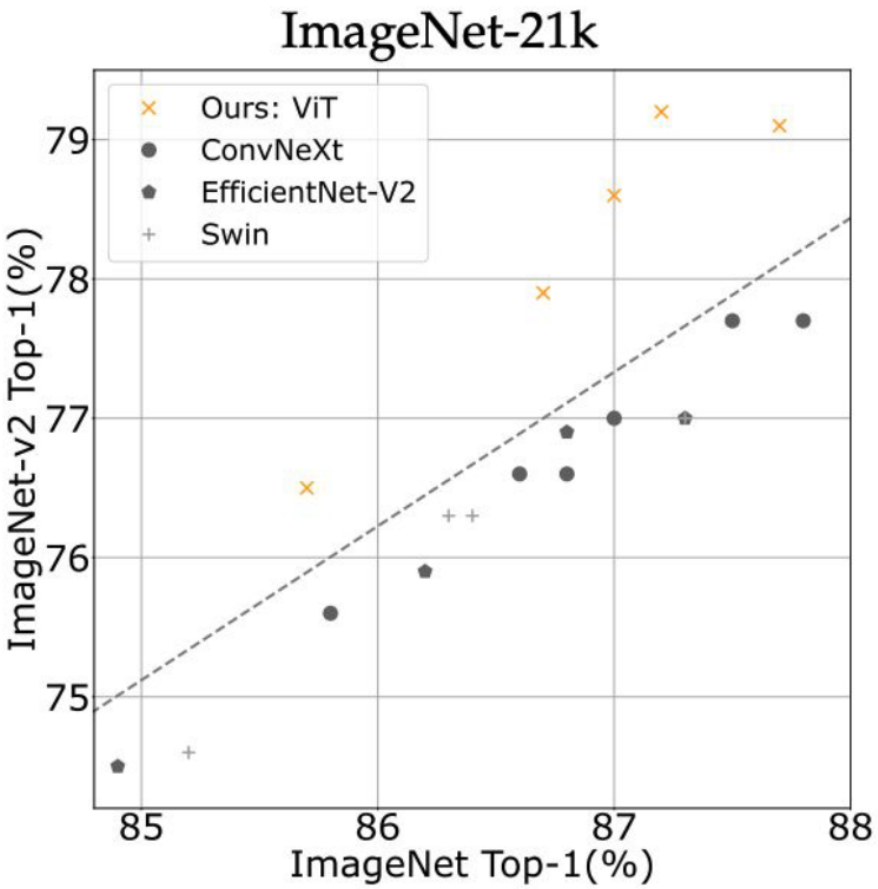


Visual Computing Institute | RWTH AACHEN UNIVERSITY

Source: [Liu et al., CVPR 2022]

DeiT III: Revenge of the ViT





Slide credit: Fei-Fei Li

Source: [Touvron et al., arXiv 2022]

Efficient Transformers

- Bottleneck
  - Computational and memory complexity $\mathcal{O}(n^2)$

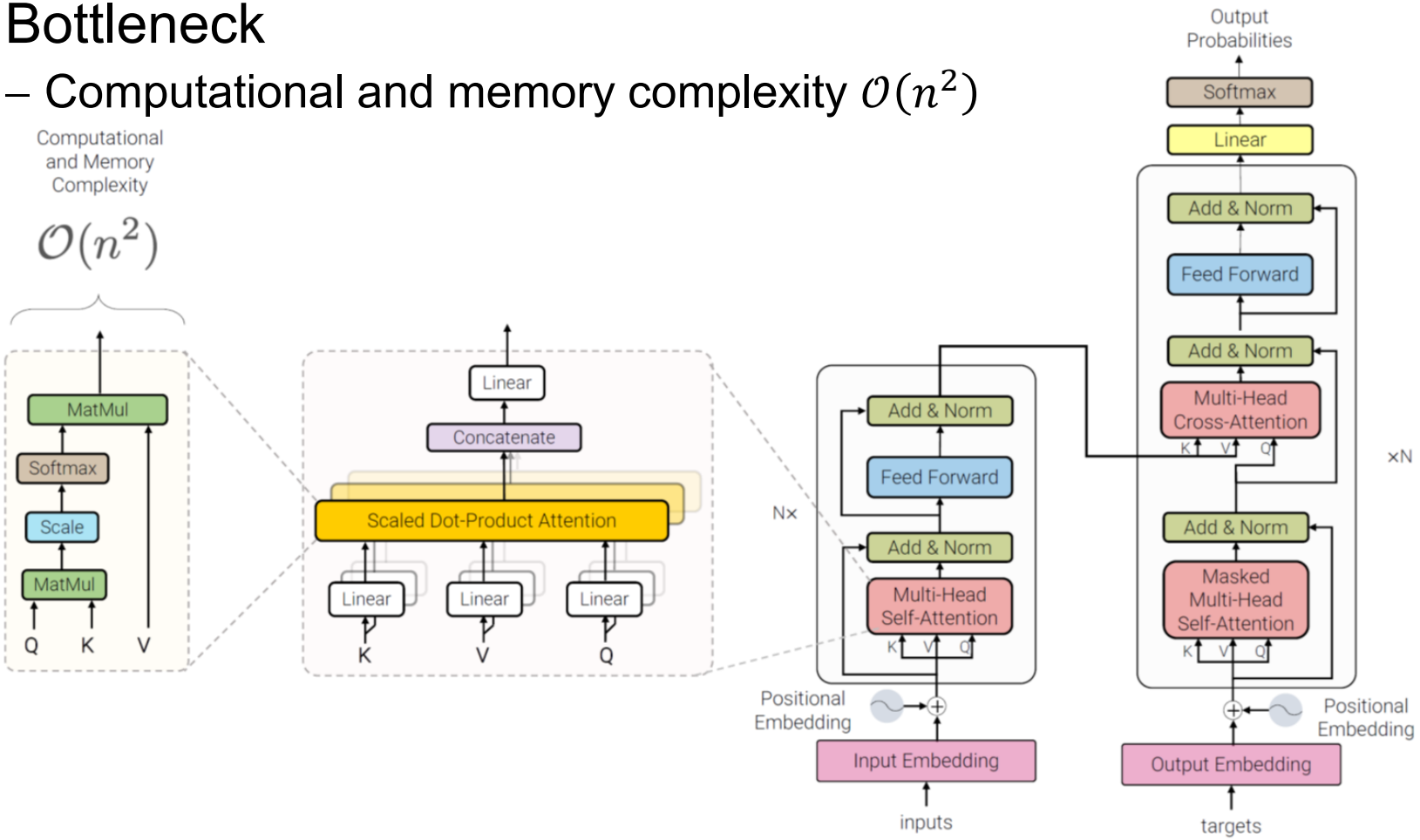


Image from [Tay et al., 2020]

Efficient Transformers

- Research directions
  - A lot of work currently happening…

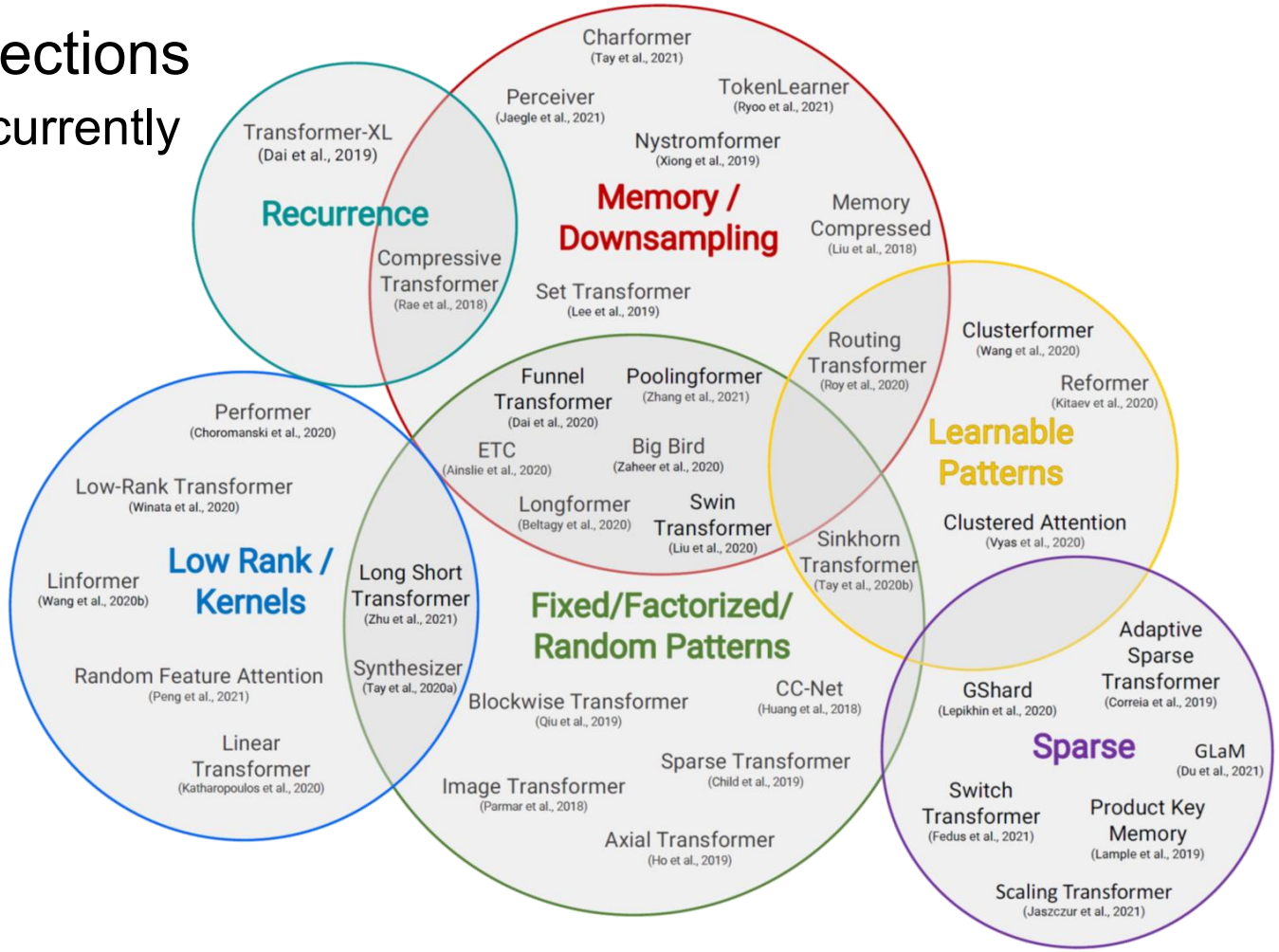


Image from [Tay et al., 2020]

Discussion: Transformers

- New way of designing deep networks
  - Adding attention to RNNs allows them to attend parts of the input at every time step
  - The general attention layer is a new type of layer used to design new NN architectures
  - Transformers are a type of layer that uses self-attention and layer norm.
    - Highly scalable and highly parallelizable
    - Faster training, larger models, better performance across vision and language tasks

Slide credit: Fei-Fei Li

Discussion: Transformers

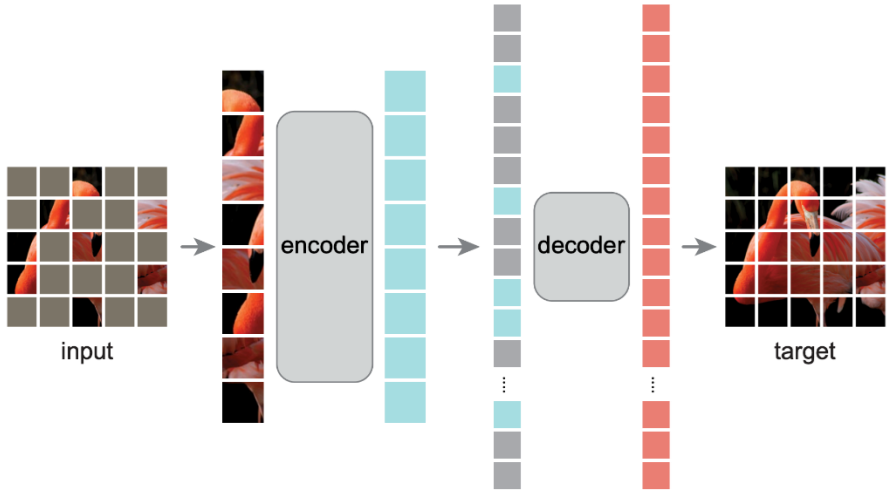
- Different computational model
  - Transformers are permutation invariant, they process sets of tokens
  - If order is relevant, it needs to be added through positional encodings
    - The transformer can then learn to use the position information if that helps it perform its task

- Multimodal Fusion becomes easier
  - Everything is a sequence of tokens
    - Text… tokens
    - An image… tokens
    - A LiDAR point cloud… tokens
  - This makes it easier to build multimodal models

Topics of This Lecture

- Recap: Transformer Architecture
  - General Attention Layer
  - Self-Attention
  - Positional Encoding
  - Original Transformer Architecture
- Transformers for Vision
  - Image captioning with Transformers
  - Vision Transformers: ViT, Swin
  - Transformer Architectures

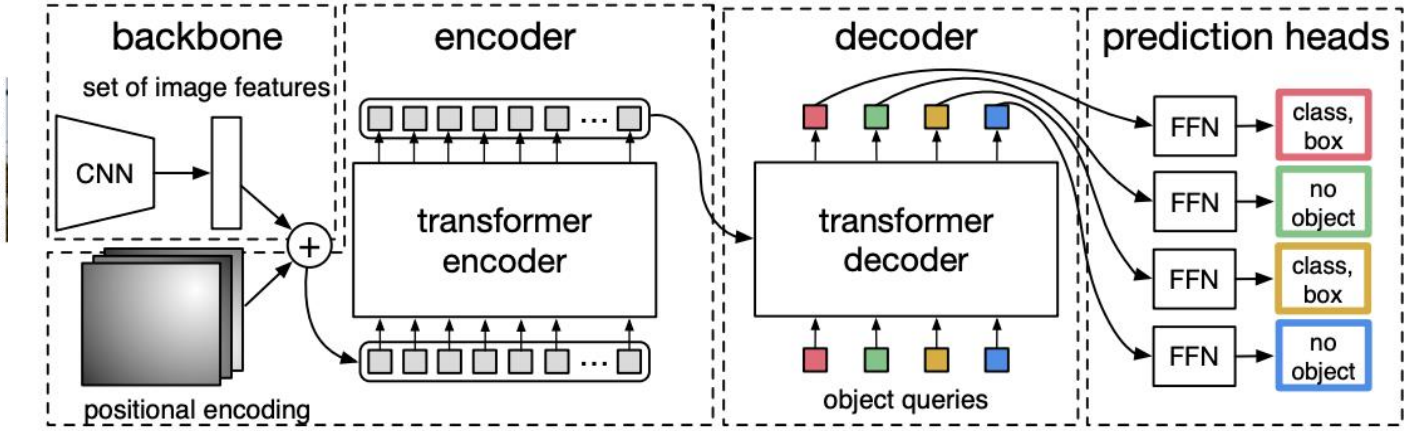
Masked Autoencoders (MAE)

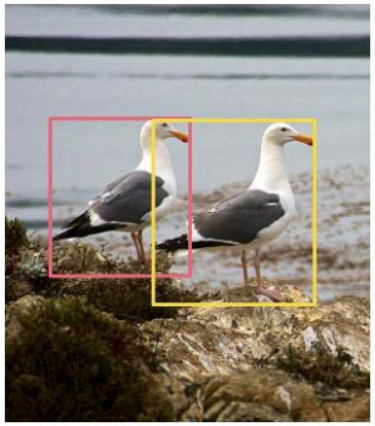


- MAE idea
  - Application of Dropout / Random Erasing idea to Transformers
  - Idea: Mask random patches of the input and reconstruct the missing pixels
    - Encoder only works on the visible subset of patches
    - Light-weight decoder to reconstruct the masked patches (thrown away after training)
    - Masking a high proportion (~75%) of the image yields a challenging and meaningful self-supervised pretraining task
    - Can easily be combined with different transformer backbones (e.g., ViT)

K. He, X. Chen, S. Xie, Y. Li, P. Dollar, Masked Autoencoders Are Scalable Vision Learners, CVPR 2022.

Transformers for Object Detection: DETR

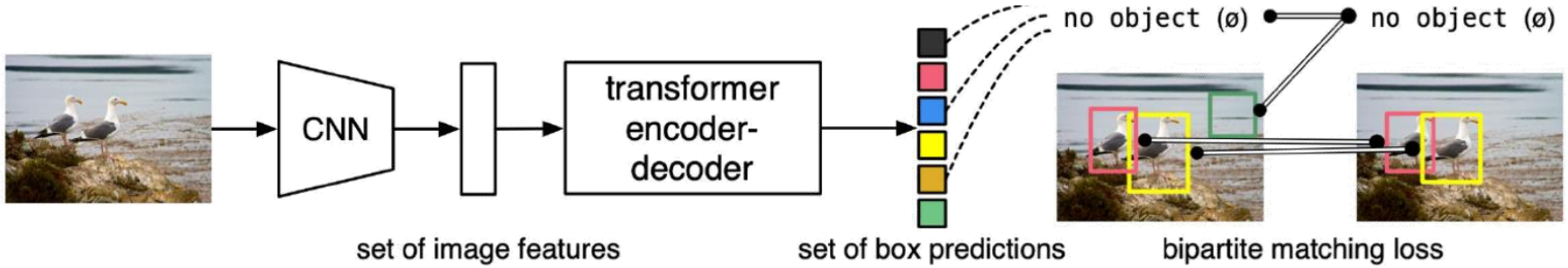




- DETR architecture
  - Conventional CNN backbone to extract 2D image features
  - Transformer encoder supplements these features with positional encodings before encoding them
  - Transformer decoder then takes a set of positional encodings as "object queries" and attends to the encoder output
  - Output embeddings are passed to shared FFN that predicts the detection (Object class + bbox prediction) or "no class".

N. Carion et al., End-to-end Object Detection with Transformers, ECCV 2020.

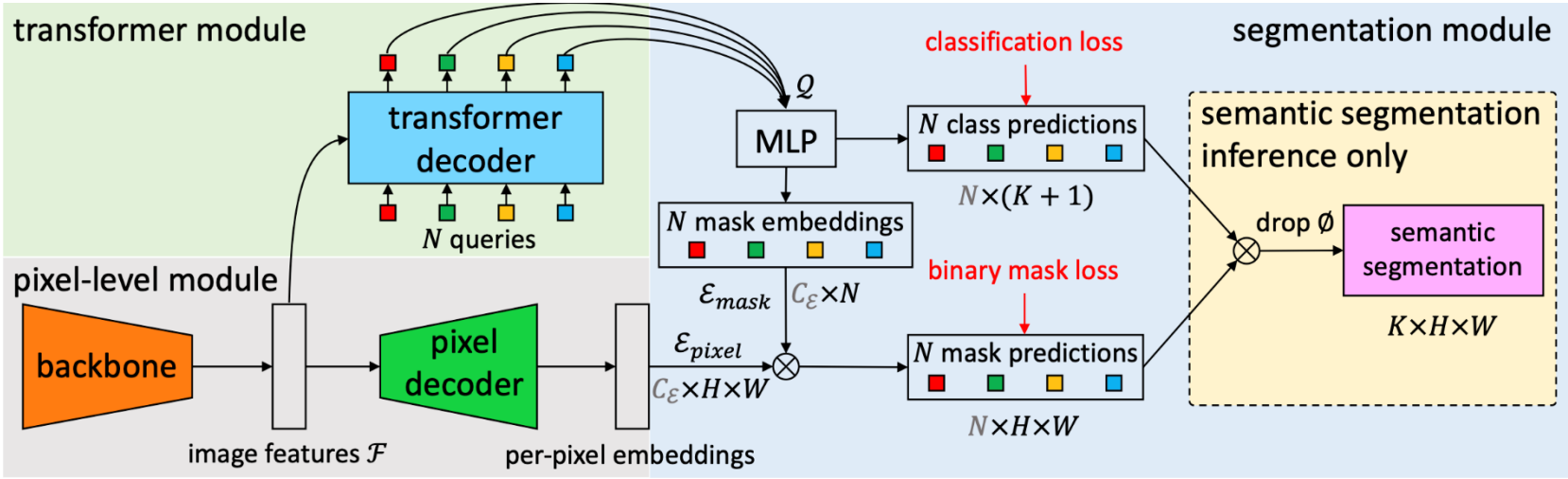
Transformers for Object Detection: DETR



- DETR properties
  - Formulation as a direct set prediction problem for an entire set of detections
    - Set-based global loss with bipartite matching to force unique predictions
  - Parallel processing of the set of N object queries
    - Very fast and efficient inference (half the computation of Faster R-CNN)
    - Number of object queries determines maximum number of detections per image
    - Default setting: $N = 100$
  - Can be extended with mask heads to output instance segmentation masks

N. Carion et al., End-to-end Object Detection with Transformers, ECCV 2020.

Transformers for Dense Prediction: MaskFormer

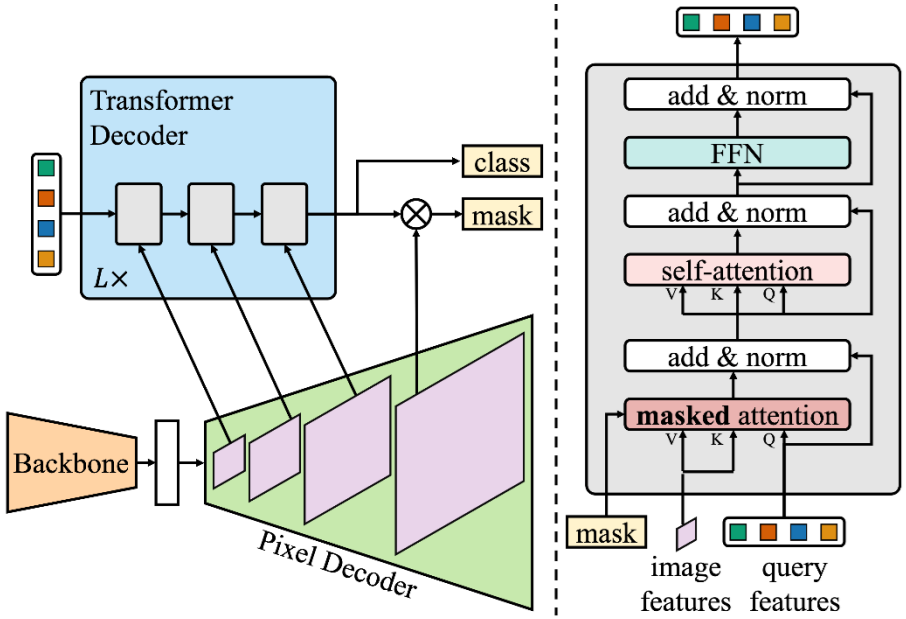


- MaskFormer ideas
  - Prior work: Segmentation considered as a per-pixel classification problem.
  - *Here*: Instead consider it as a mask classification problem.
    - Predict a set of binary masks
    - Each mask is associated with a single global class label

  ⇒ Unifies *semantic segmentation*, *instance segmentation*, and *panoptic segmentation*

B. Cheng, I. Misra, A. Schwing, A. Kirillov, A. Girdhar, Masked-Attention Mask Transformer for Universal Image Segmentation, CVPR 2022.

Extension: Mask2Former

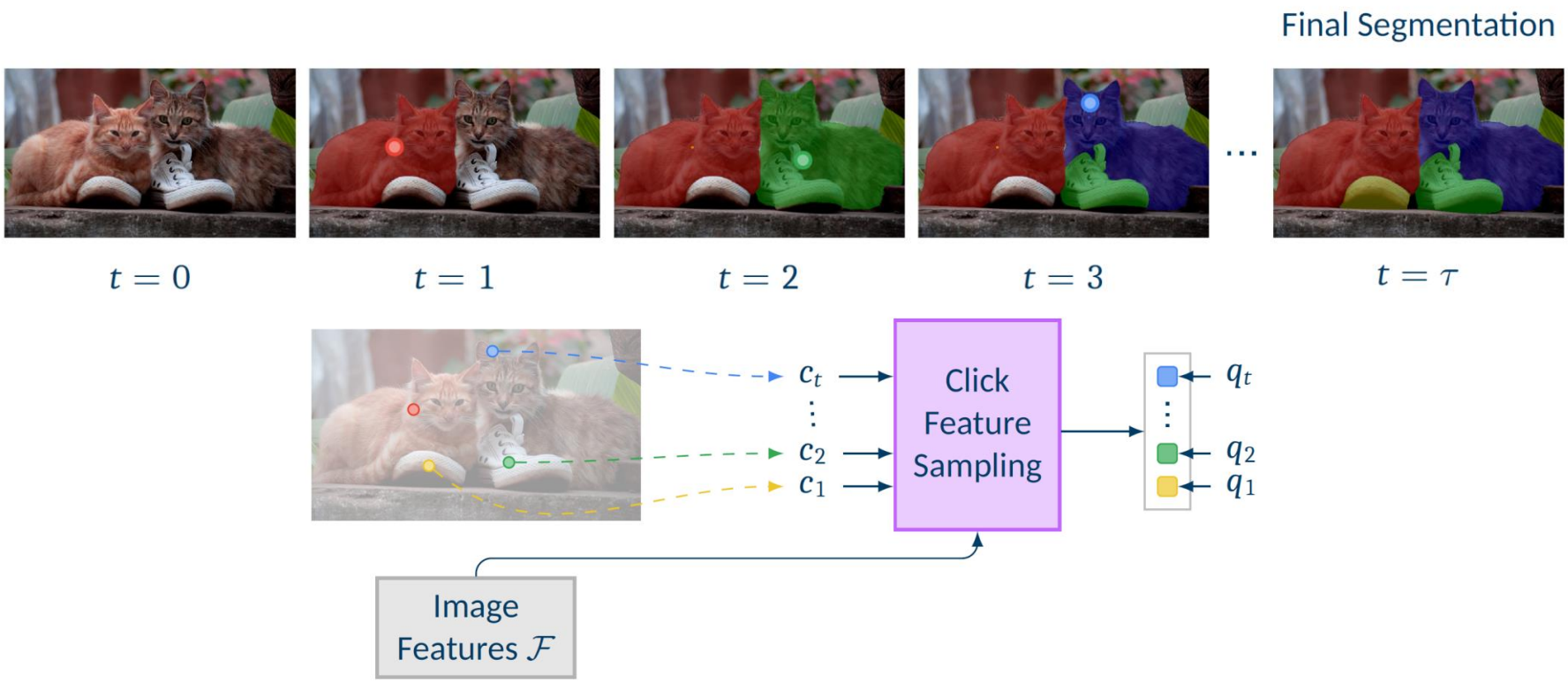


- Mask2Former
  - Same meta architecture as MaskFormer
  - Transformer decoder now contains a masked attention operator
    - Extracts more localized features by constraining cross-attention to within the foreground region of the predicted mask
  - Multi-scale processing using pixel decoder's feature pyramid
    - Better handling of small objects

Example: Interactive Segmentation

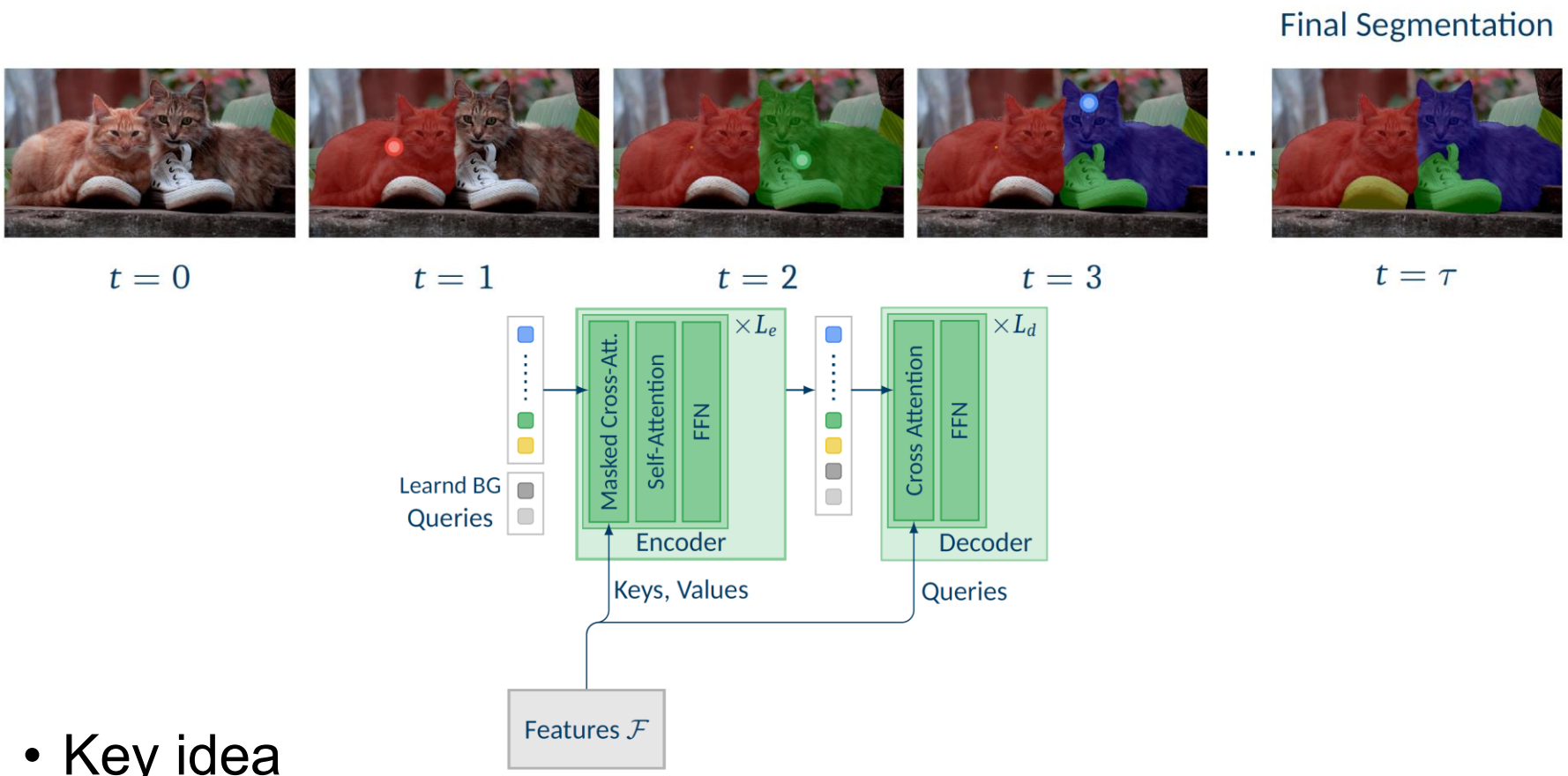
[Mahadevan, Rana, Hermans, Leibe, ICCV'23]

Clicks as Spatiotemporal Sequence



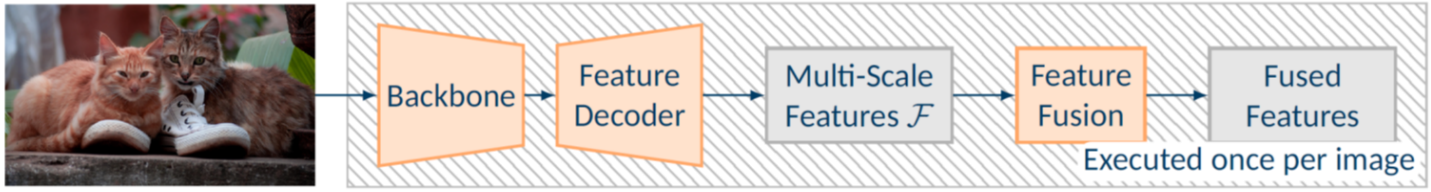
- Key idea
  - Reformulate clicks as spatiotemporal queries

Clicks as Spatiotemporal Sequence



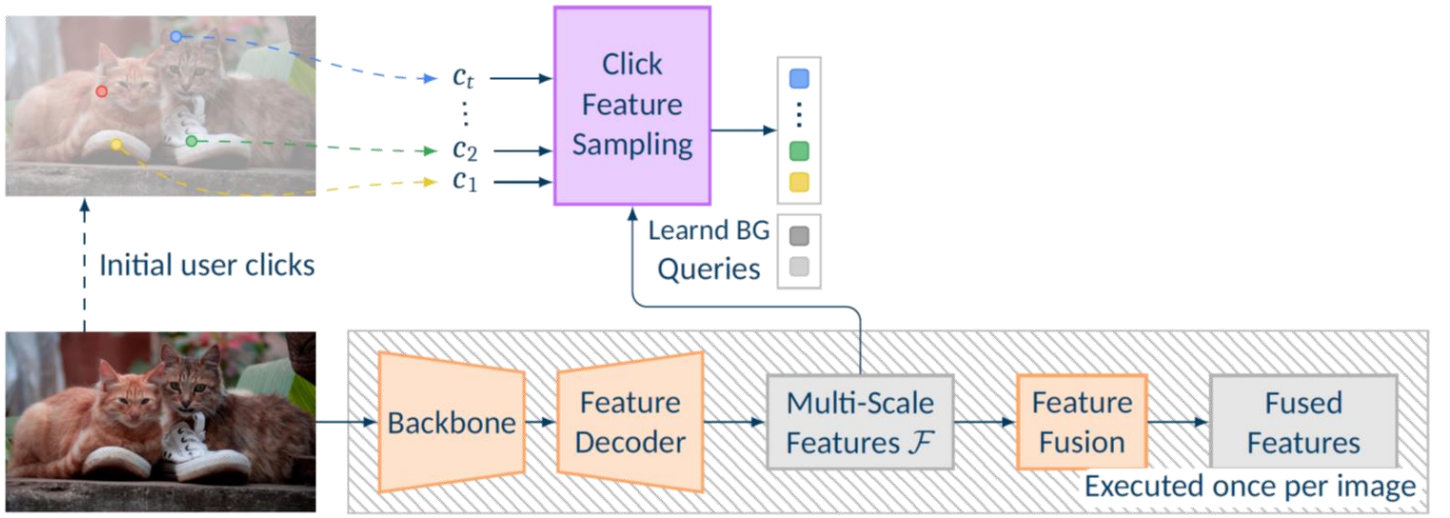
- Key idea
  - Reformulate clicks as spatiotemporal queries processed by a Mask Transformer module

DynaMITe Architecture



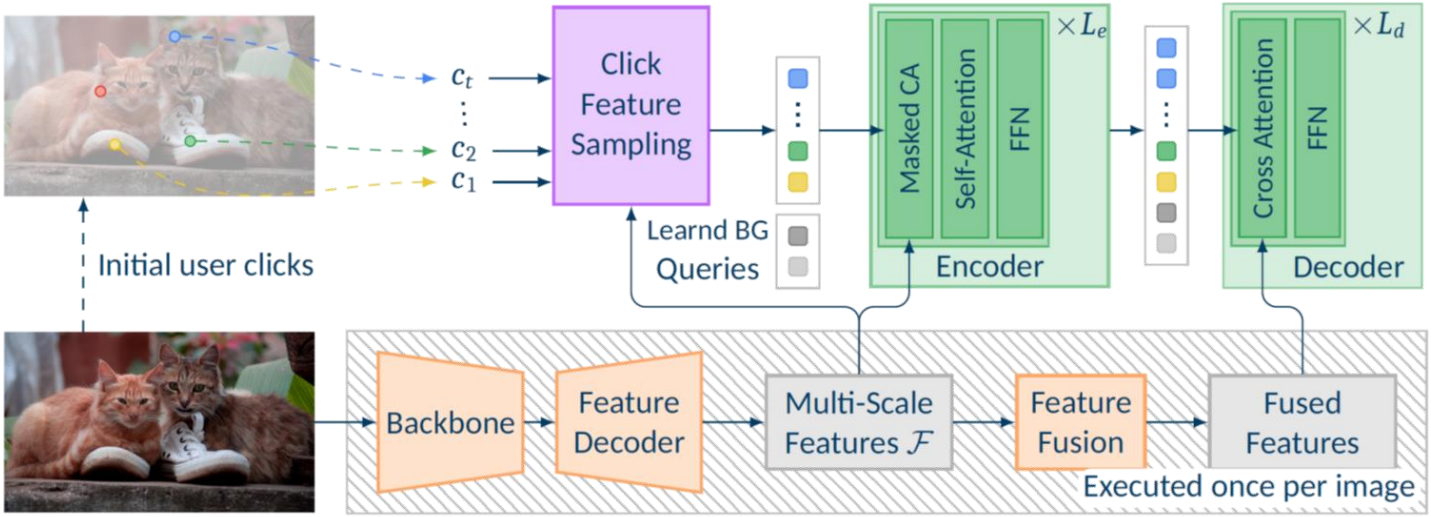
[Mahadevan, Rana, Hermans, Leibe, ICCV'23]

DynaMITe Architecture



[Mahadevan, Rana, Hermans, Leibe, ICCV'23]

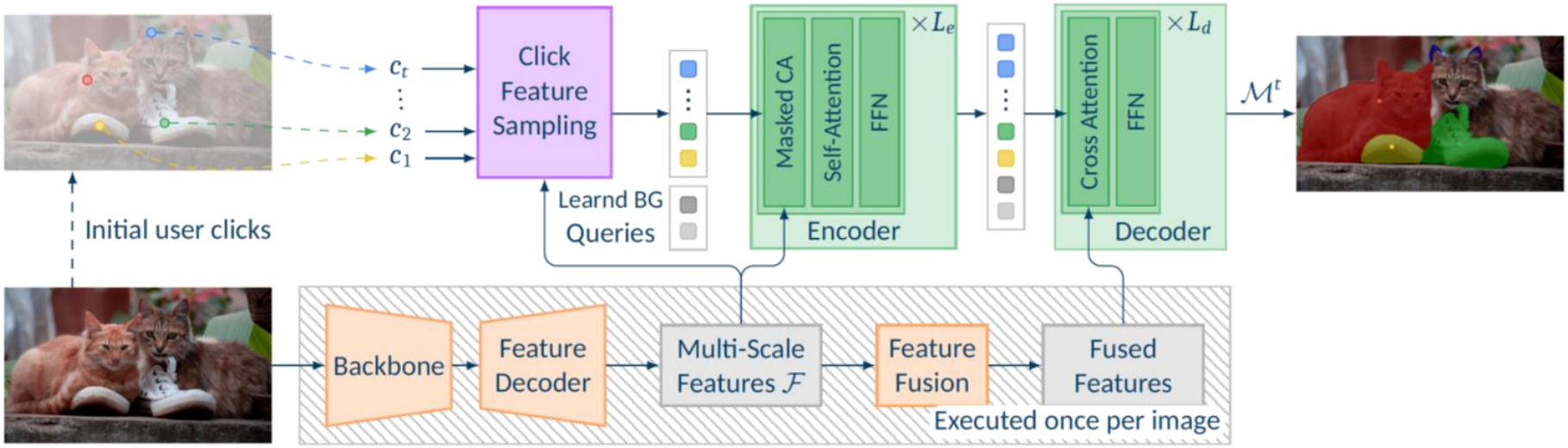
DynaMITe Architecture



Visual Computing Institute | RWTH AACHEN UNIVERSITY

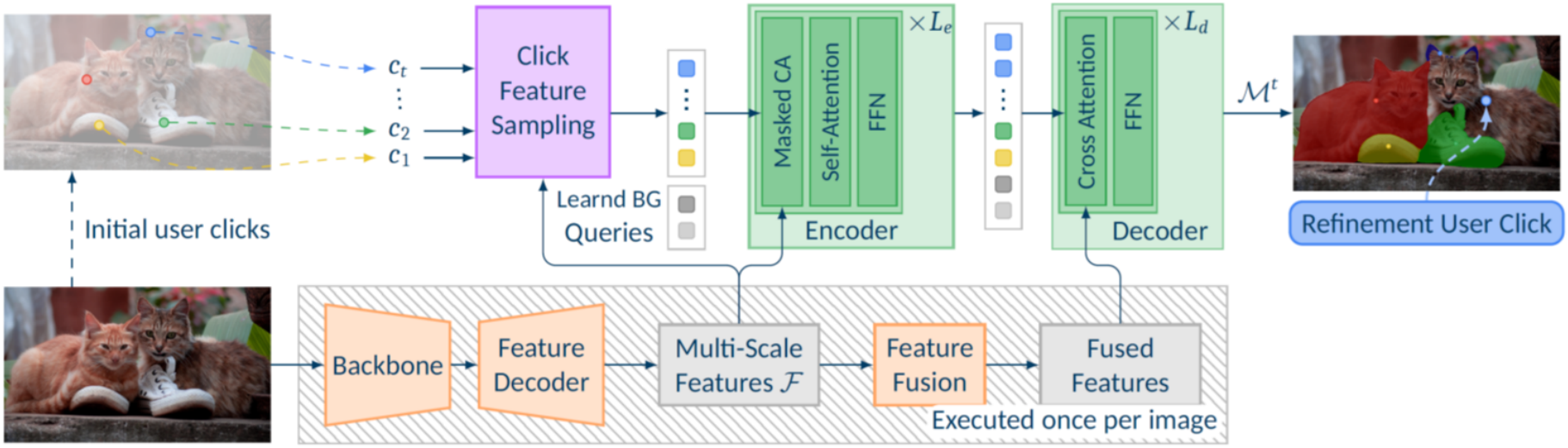
[Mahadevan, Rana, Hermans, Leibe, ICCV'23]

DynaMITe Architecture



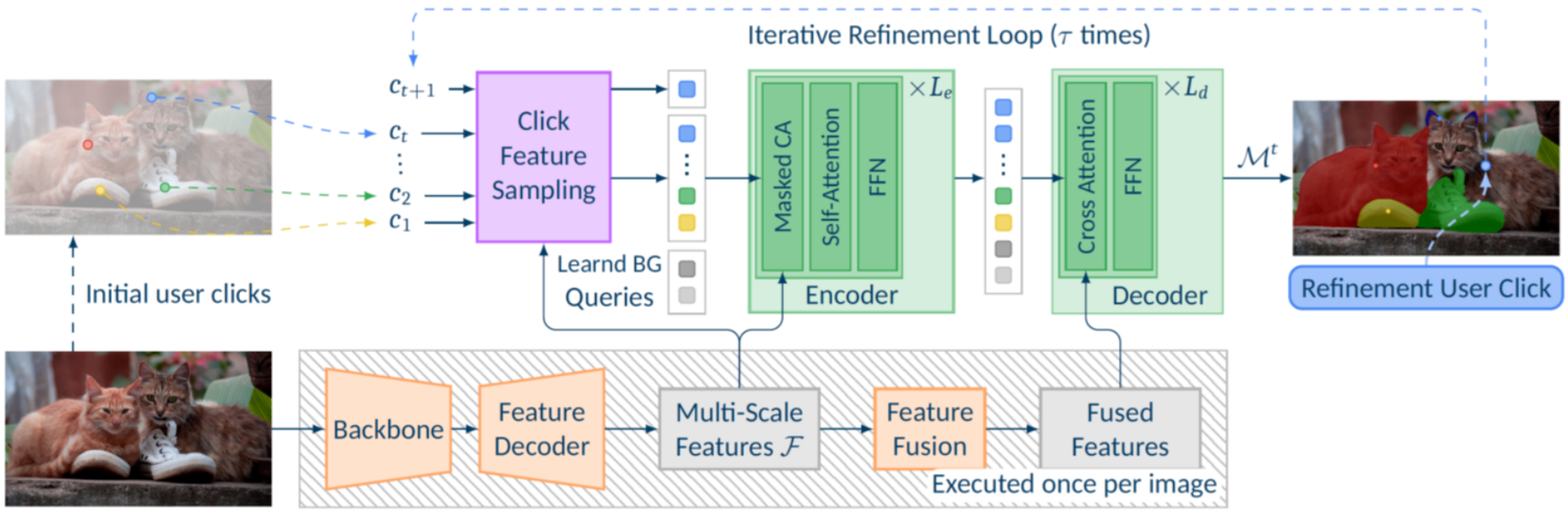
[Mahadevan, Rana, Hermans, Leibe, ICCV'23]

DynaMITe Architecture



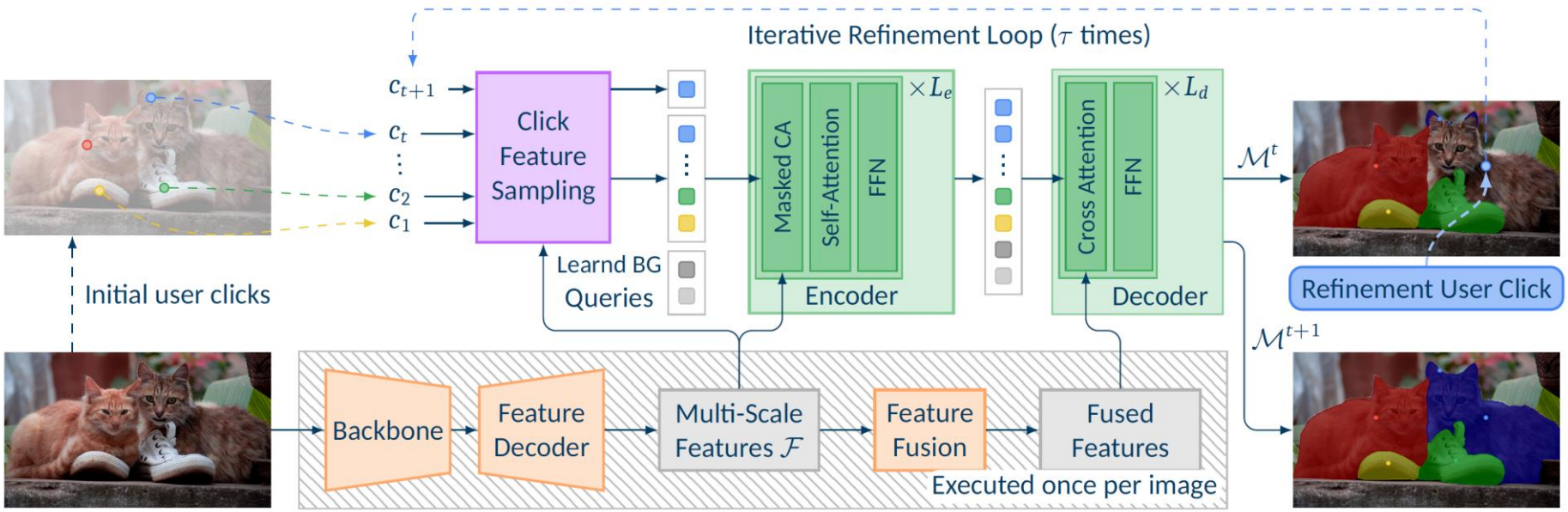
[Mahadevan, Rana, Hermans, Leibe, ICCV'23]

DynaMITe Architecture



[Mahadevan, Rana, Hermans, Leibe, ICCV'23]

DynaMITe Architecture

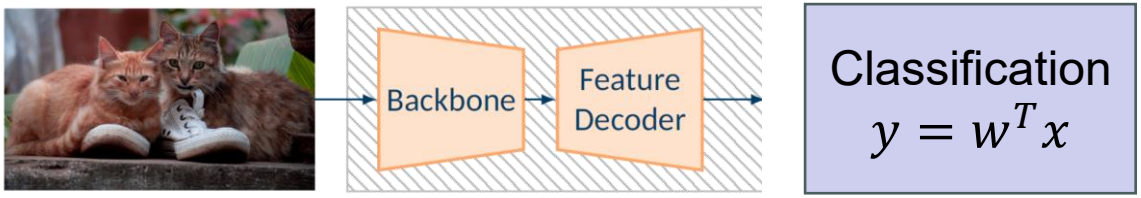


- Advantages
  - Designed for multi-instance segmentation from the start
  - Image-level backbone features need to be computed only once
  - Reduces required #clicks through common background representation

[Mahadevan, Rana, Hermans, Leibe, ICCV'23]

Why Does This Work?

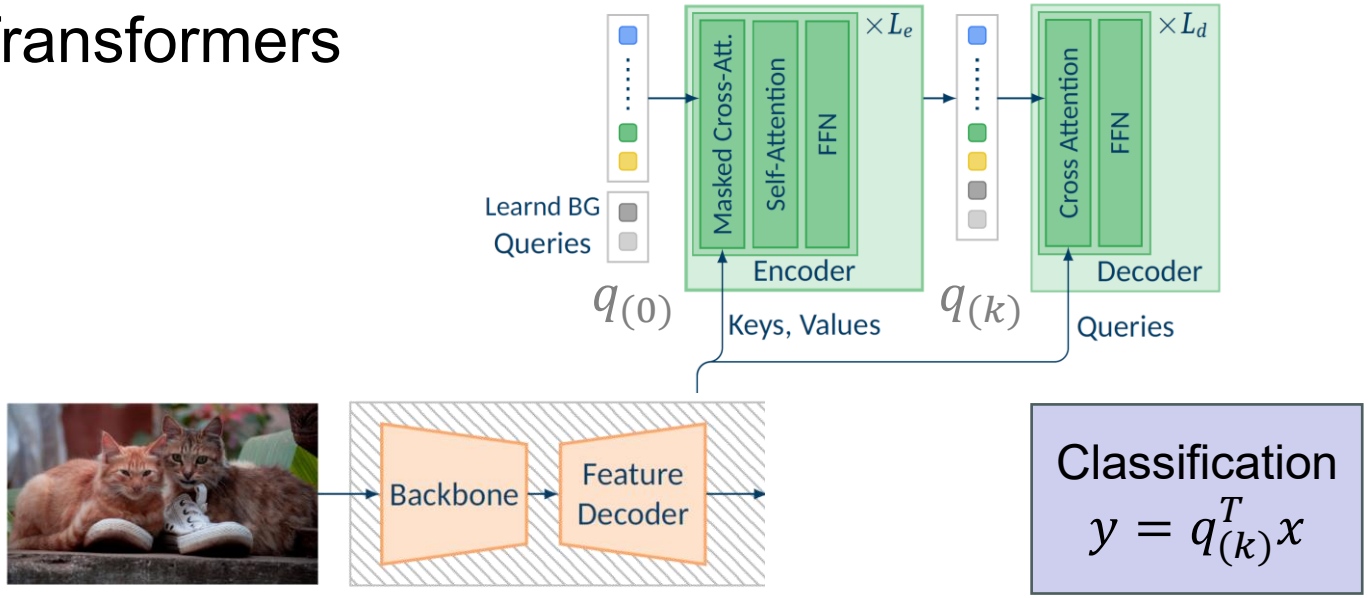
- Traditional CNNs



- What they do
  - Build a representation w for the concept “cat” and compare all pixel features to this representation.
  - w is fixed at training time

Why Does This Work?

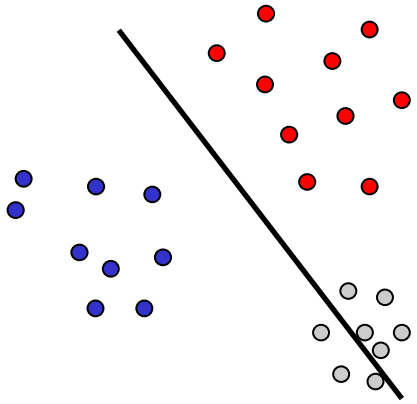
- Mask Transformers



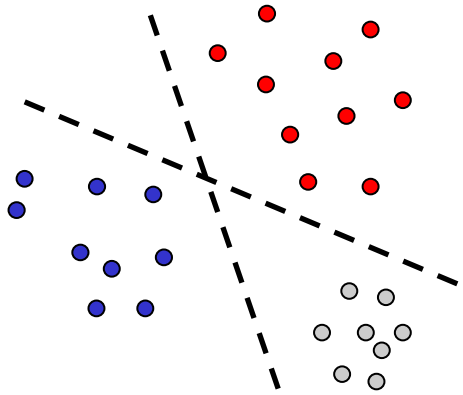
- What they do
  - Start from an initial query $q_{(0)}$ and successively adapt it $q_{(0)} \longrightarrow q_{(k)}$ *to the specific image at test time*, taking into account the other queries
  - They thus ask "What does it mean to 'segment the brown cat' *in the context of this image?*"

Comparison

Classification
$y = w^T x$



Classification
$y = q_{(k)}^T x$



- Classic CNNs
  - Decision boundary is fixed at training time
  - Significant training effort needed to get it “always right”
- Mask Transformers
  - Decision boundary can be adjusted at test time, based on context
  - Some indication that this also helps regularize learning

References

- Transformers
  - A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is All You Need. NIPS 2017.
  - H. Zhang, I. Goodfellow, D. Metaxas, A. Odena. Self-Attention Generative Adversarial Networks. ICML 2018.